

A Mini Project Report on

SmartFuel-Fuel Consumption Prediction Model Using Machine Learning

Submitted in partial fulfillment for award of the degree of Bachelor of Technology in
Computer Science and Engineering (Artificial Intelligence & Machine Learning)

by

K.Amithi	22321A6605
B.Ananya	22321A6606
A.Chakrika	22321A6610
B.Deepa	22321A6611

Under the esteemed Guidance of

Internal Guide

Mr Mohd Basit Mohiuddin,

Assistant Professor, CSE(AI&ML)



Bhoj Reddy Engineering College for Women

Department of Computer Science and Engineering(Artificial Intelligence & Machine Learning)

(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade,

Approved by AICTE and Affiliated to JNTUH)

Recognised by UGC under section 2(f) of the UGC Act, 1956.

Vinaynagar, IS Sadan Crossroads, Hyderabad – 500 059

Ph: +91-40-2453 7282; Website: www.brecw.ac.in; Email: principal@brecw.ac.in

2024- 2025



Bhoj Reddy Engineering College for Women
(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade,
Approved by AICTE and affiliated to JNTUH, Recognized by UGC under section 2(f) of the UGC Act, 1956)
Vinaynagar, IS Sadan Crossroads, Saidabad, Hyderabad – 500 059, Telangana. www.brecw.ac.in



Date:09-06-2025

CERTIFICATE

This is to certify that the Mini Project Report entitled “**SmartFuel-Fuel Consumption Prediction Model Using Machine Learning**” is a bonafide work carried out by

K.Amithi	22321A6605
B.Ananya	22321A6606
A.Chakrika	22321A6610
B.Deepa	22321A6611

in partial fulfillment for award of the degree of Bachelor of Technology in Computer Science and Engineering(Artificial Intelligence & Machine Learning) department from Bhoj Reddy Engineering College for Women, Hyderabad, affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH).

Internal Guide

Mr Mohd Basit Mohiuddin

Head of the Department

Dr. M Seshu Bhavani

External Examiner



Bhoj Reddy Engineering College for Women

(Sponsored by Sangam Laxmibai Vidyapeet, Accredited by NAAC with A Grade,
Approved by AICTE and affiliated to JNTUH, Recognized by UGC under section 2(f) of the UGC Act, 1956)
Vinaynagar, IS Sadan Crossroads, Saidabad, Hyderabad – 500 059, Telangana. www.brecw.ac.in



Date:09-06-2025

CERTIFICATE

This is to certify that the Mini Project Report entitled “**SmartFuel-Fuel Consumption Prediction Model Using Machine Learning**” is a bonafide work carried out by

K.Amithi	22321A6605
B.Ananya	22321A6606
A.Chakrika	22321A6610
B.Deepa	22321A6611

in partial fulfillment for award of the degree of Bachelor of Technology in Computer Science and Engineering(Artificial Intelligence & Machine Learning) department from Bhoj Reddy Engineering College for Women, Hyderabad, affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH).

Internal Guide

Mr Mohd Basit Mohiuddin

Head of the Department

Dr. M Seshu Bhavani

ACKNOWLEDGMENT

It is our pleasure to express our whole-hearted thanks to M Seshu Bhavani, Associate Professor & Head of Computer Science and Engineering (Artificial Intelligence & Machine Learning)

We are thankful to our Department, for providing the necessary facilities to undergo this project.

Project coordinator Mohd Basit Mohiuddin, Assistant Professor, Computer Science and Engineering (Artificial Intelligence & Machine Learning) Department, for giving us this opportunity to do this project.

We are also thankful to our internal guide Mr Mohd Basit Mohiuddin,, Assistant Professor, Computer Science and Engineering (Artificial Intelligence & Machine Learning) Department, for her extreme guidance and support in completing this project successfully.

We are also thankful to the staff members of the Computer Science and Engineering (Artificial Intelligence & Machine Learning) Department, my friends and to our parents who helped us in completing this project successfully.

By
KAmithi (22321A6605)

B.Ananya 22321A6606)

A.Chakrika (22321A6610)

B.Deepa (22321A6611)

Contents

	Page No
List of Figures	i
Abstract	ii
1. Introduction	1-3
1.1 Introduction of the project	2
1.2 Existing system	2
1.3 Problems in existing system	3
1.4 Proposed system	3
1.5 Advantages of proposed system	3
2. Requirements Analysis	4-6
2.1 Functional requirements	5
2.2 Non-functional requirements	5
2.3 Computational resource requirements	6
2.3.1 Hardware resources	6
2.3.2 Software resources	6
3. Design	7-15
3.1 Software Achitecture	8
3.2 Software Process Model	9
3.3 UML Diagrams	9
3.3.1 Use Case Diagram	10
3.3.2 Class Diagram	11
3.3.3 Activity Diagram	12
3.3.4 Activity Diagram	13
3.3.5 Sequence Diagram	14
3.3.6 Collaboration Diagram	15

4. Implementation	16-21
4.1 Technologies	17
4.2 Pseudocode	19
5. Testing	22-27
5.1 Overview	23
5.2 Dimensions of Testing	23
5.3 Stages of Testing	23-25
5.3.1 Unit Testing	23
5.3.2 Integration Testing	24
5.3.4 System Testing	24
5.3.5 Acceptance Testing	25
5.4 Types of Testing	25-26
5.5 Test Cases	26-27
6. Screenshots	28-31
7. Conclusion and Future Scope	32-33
7.1 Conclusion	31
7.2 Future Scope	31
8. References	34-35

List of Figures

Fig.No	Figure Name	Page.No
3.1	System Architecture	8
3.2	Software Process Model	9
3.1	Use Case Diagram	10
3.2	Class Diagram	11
3.3	Activity Diagram	12
3.4	Activity Diagram	13
3.5	Sequence Diagram	14
3.6	Collaboration Diagram	15
Screenshots		28-31
6.1	Activate Project	28
6.2	Main Page	28
6.3	Fuel Efficient Cars	29
6.4	Sports Cars	29
6.5	Family Cars	30
6.6	Output 1	30
6.7	Output 2	31

Abstract

In this project, we develop a machine learning model to predict vehicle fuel consumption using the Logistic Regression algorithm. The aim is to enhance the accuracy of fuel consumption predictions to achieve better fuel efficiency, leading to economic benefits and meeting industry needs. The proposed model considers key features such as Mass Air Flow, Vehicle Speed, RPM, and Throttle Position Sensor to estimate fuel consumption. Our model is trained and tested on a comprehensive On-Board Diagnostics dataset that includes 18 features. By leveraging Logistic Regression, we provide a straightforward, interpretable, and efficient approach to predicting fuel consumption. The results demonstrate the model's capability to deliver reliable predictions, offering an alternative to more complex algorithms. This project underscores the potential of Logistic Regression in creating practical and effective fuel consumption prediction models, aiding manufacturers in developing strategies to minimize fuel usage and improve vehicle performance.

INTRODUCTION

1. Introduction

1.1 Introduction of the project

Aims to develop a robust fuel consumption prediction model using Random Forest, focusing on its simplicity, interpretability, and efficiency. Random Forest is chosen for its ability to model binary outcomes effectively, making it suitable for predicting vehicle fuel consumption. Random Forest transforms input features into probabilities using a sigmoid function, enabling straightforward interpretation of predictions. The system emphasizes optimizing fuel efficiency and economic benefits by accurately estimating fuel consumption. By leveraging Random Forest strengths in providing probabilistic outputs, the proposed system aims to deliver reliable predictions that can assist manufacturers in making informed decisions to improve vehicle performance and reduce operational costs.

1.2 Existing System

Support Vector Machines (SVM) in predicting vehicle fuel consumption, focusing on its effectiveness in handling complex relationships between input features. SVM is employed to model the relationship between variables such as Mass Air Flow, Vehicle Speed, Revolutions Per Minute, and Throttle Position Sensor, leveraging its ability to find optimal hyperplanes for classification tasks. These systems typically utilize SVM's kernel methods, including polynomial, radial basis function (RBF), and sigmoid kernels, to transform data into higher dimensional spaces where linear separation of classes is feasible. By maximizing the margin between support vectors and the decision boundary, SVM enhances the accuracy of fuel consumption predictions

1.3 Problems in Existing System

- 1) Especially with large datasets, making it slower to train compared to simpler algorithms.
- 2) Requires significant memory for large datasets, as it needs to store all support vectors.

1.4 Proposed System

Aims to develop a robust fuel consumption prediction model using Random Forest, focusing on its simplicity, interpretability, and efficiency. Random Forest is chosen for its ability to model binary outcomes effectively, making it suitable for predicting vehicle fuel consumption. Random Forest transforms input features into probabilities using a sigmoid function, enabling straightforward interpretation of predictions. The system emphasizes optimizing fuel efficiency and economic benefits by accurately estimating fuel consumption. By leveraging Random Forest strengths in providing probabilistic outputs, the proposed system aims to deliver reliable predictions that can assist manufacturers in making informed decisions to improve vehicle performance and reduce operational costs.

1.5 Advantages of Proposed System

It combines multiple decision trees to reduce overfitting and improve accuracy.

- It handles missing data and outliers effectively.
- Works for both classification and regression tasks.
- Can be easily parallelized, speeding up training on large datasets

REQUIREMENT ANALYSIS

2. Requirement Analysis

2.1 Functional Requirements

Data Input Module: The system must allow the upload of vehicle data including On-Board Diagnostics (OBD) parameters from a legacy dataset (e.g., CSV format).

Data Preprocessing: The system should clean, normalize, and handle missing values in the dataset to prepare it for model training and prediction.

Model Training: The system must use machine learning algorithms (e.g., Linear Regression, Random Forest, etc.) to train a predictive model on historical fuel consumption data.

Fuel Consumption Prediction: The system should predict fuel consumption based on given input conditions (e.g., engine load, speed, ambient temperature, etc.).

User Interface: A web interface should allow users to input data manually or upload files and view the predicted fuel consumption results.

Visualization Module: The system must display charts or graphs (e.g., predicted vs. actual fuel consumption) for better understanding and comparison.

Model Evaluation: The system should calculate accuracy metrics (e.g., MAE, RMSE) to evaluate model performance.

Report Generation: The system must generate and download a report summarizing the input data, prediction results, and model performance.

Environmental Impact Estimation: Based on fuel consumption predictions, the system may estimate CO₂ emissions and suggest potential reductions.

2.2 Non-Functional Requirements

Performance: The system must generate fuel consumption predictions within a few seconds to ensure a smooth user experience.

Usability: The user interface should be simple, user-friendly, and accessible to users with minimal technical knowledge.

Reliability: The system must consistently provide accurate predictions and handle invalid or incomplete inputs gracefully.

Security: Uploaded datasets and user inputs must be protected from unauthorized access and data breaches.

Maintainability: The code should be well-structured and documented to support future updates or debugging with ease

2.3 Computational Resource Requirements

2.3.1 Hardware Requirements

Processor	:	I5 processor
RAM	:	Minimum 8GB
Storage	:	256GB SSD

2.3.2 Software Requirements

Operating System: Windows 11

Language: Python, HTML

Integrated Development Environment (IDE): VS Code,

Front end Technologies: HTML/CSS (Bootstrap)

DESIGN

3 DESIGN

- Design represents the number of components we are using as a part of the project and the flow of request processing i.e., what components in processing the request and in which order.
- An architecture description is a formal description and representation of a system organized in a way that supports reasoning about the structure of the system.

3.1 System Architecture

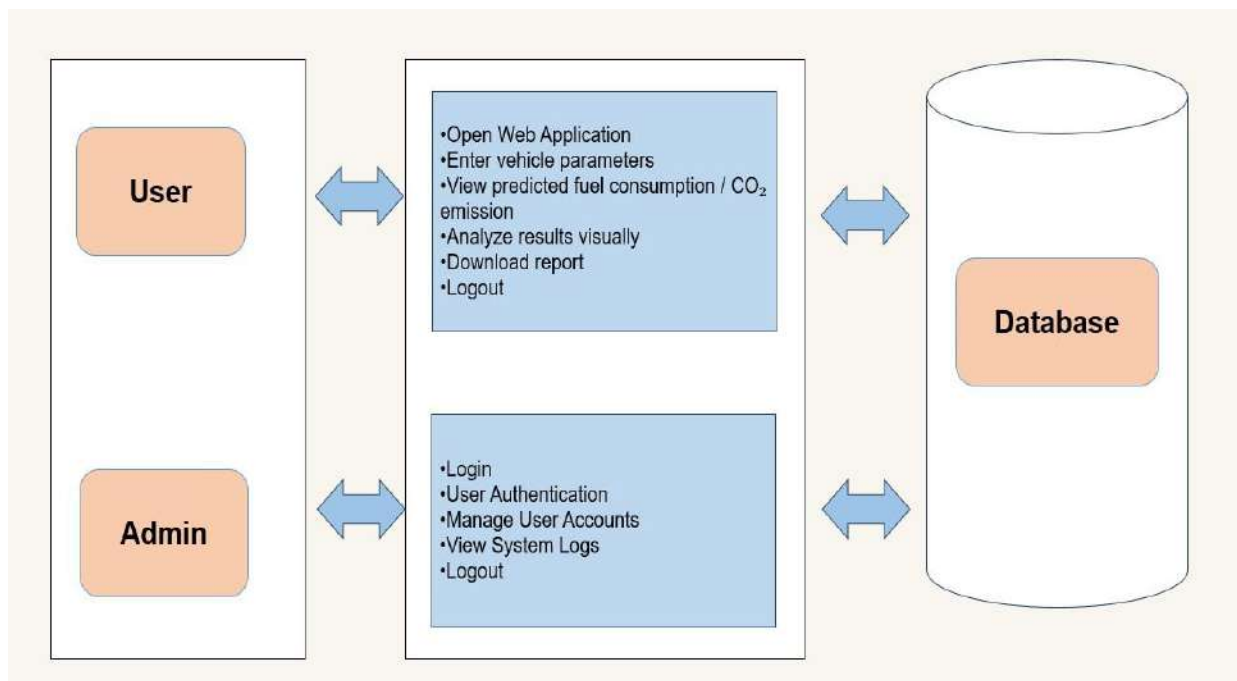


Fig 3.1 System Architecture

3.2 Software Process Model

Waterfall Model is a classical software development methodology that was first introduced by Winston W. Royce in 1970. It is a linear and sequential approach to software development that consists of several phases that must be completed in a specific order.



Fig 3.2 Software process model

3.3 UML Diagrams

UML (Unified Modeling Language) is a standardized modeling language used in software engineering to visualize, specify, construct, and document the components and design of a software system. It provides a set of graphical notations and diagrams that help developers, analysts, and stakeholders understand the architecture, behavior, and interactions within a system.

3.3.1 Use Case Diagram

A Use Case Diagram is a high-level representation of the interactions between the user (actor) and the system. It shows what functionalities the system provides without showing how these functionalities are implemented.

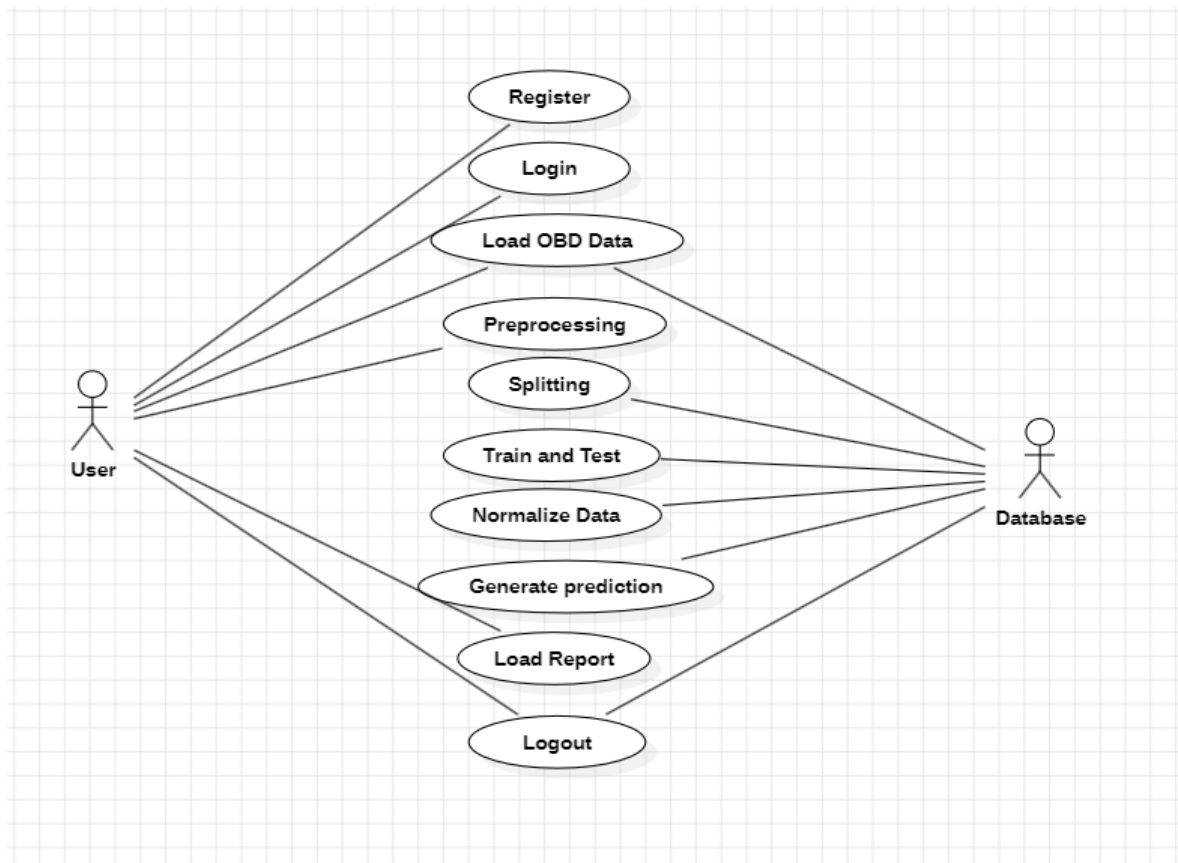


Fig 3.3 Use Case Diagram

3.3.2 Class Diagram

A Class Diagram represents the static structure of the system, showing the system's classes, their attributes, methods, and relationships.

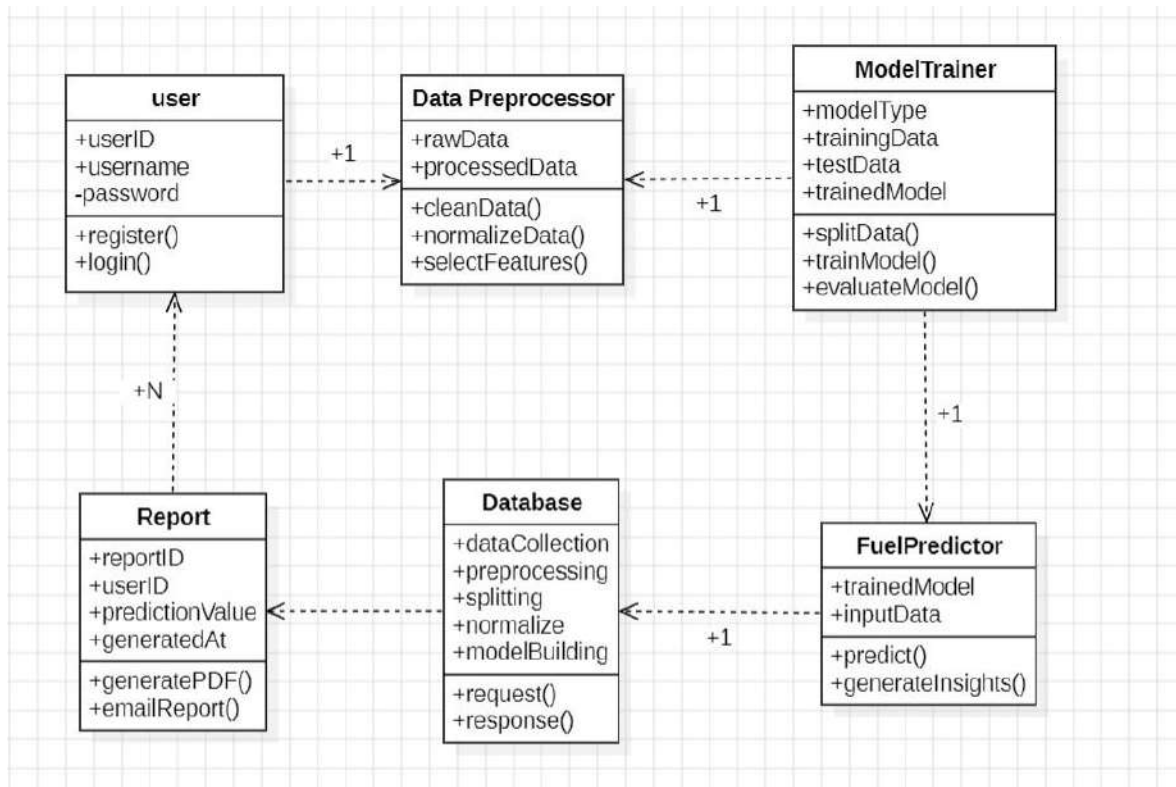


Fig 3.4 Class Diagram

3.3.3 Activity Diagram

An Activity Diagram models the workflow or step-by-step activities involved in a process. It helps visualize the flow of control from one activity to another.

To show the step-by-step flow of how an email is taken, processed, and classified by the system.

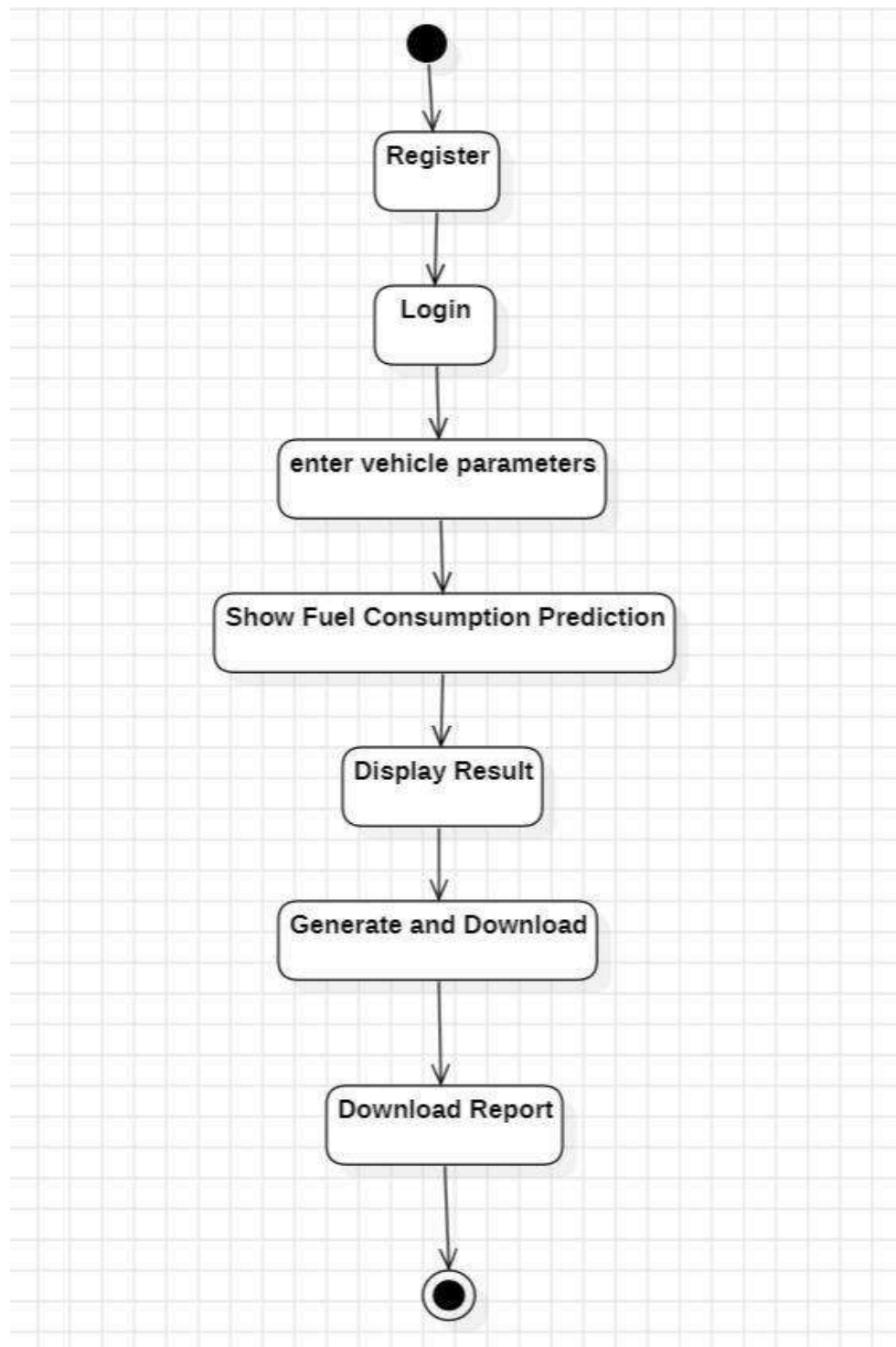


Fig 3.5 Activity Diagram for User

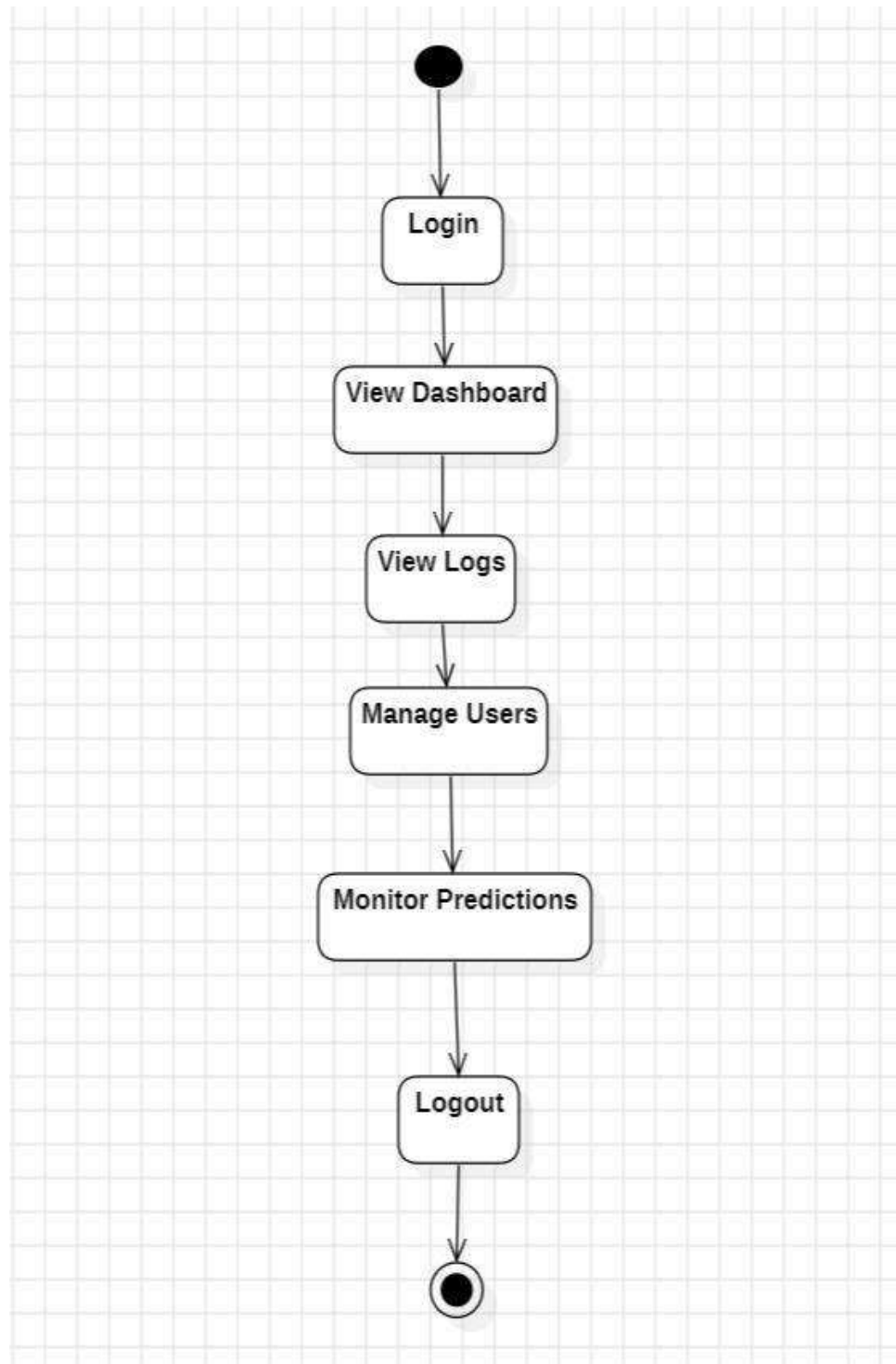


Fig 3.6 Activity Diagram for Admin

3.3.4 Sequence Diagram

A Sequence Diagram is a type of UML (Unified Modeling Language) diagram that shows how objects interact in a particular sequence over time. It focuses on the order of messages exchanged between components in a system.

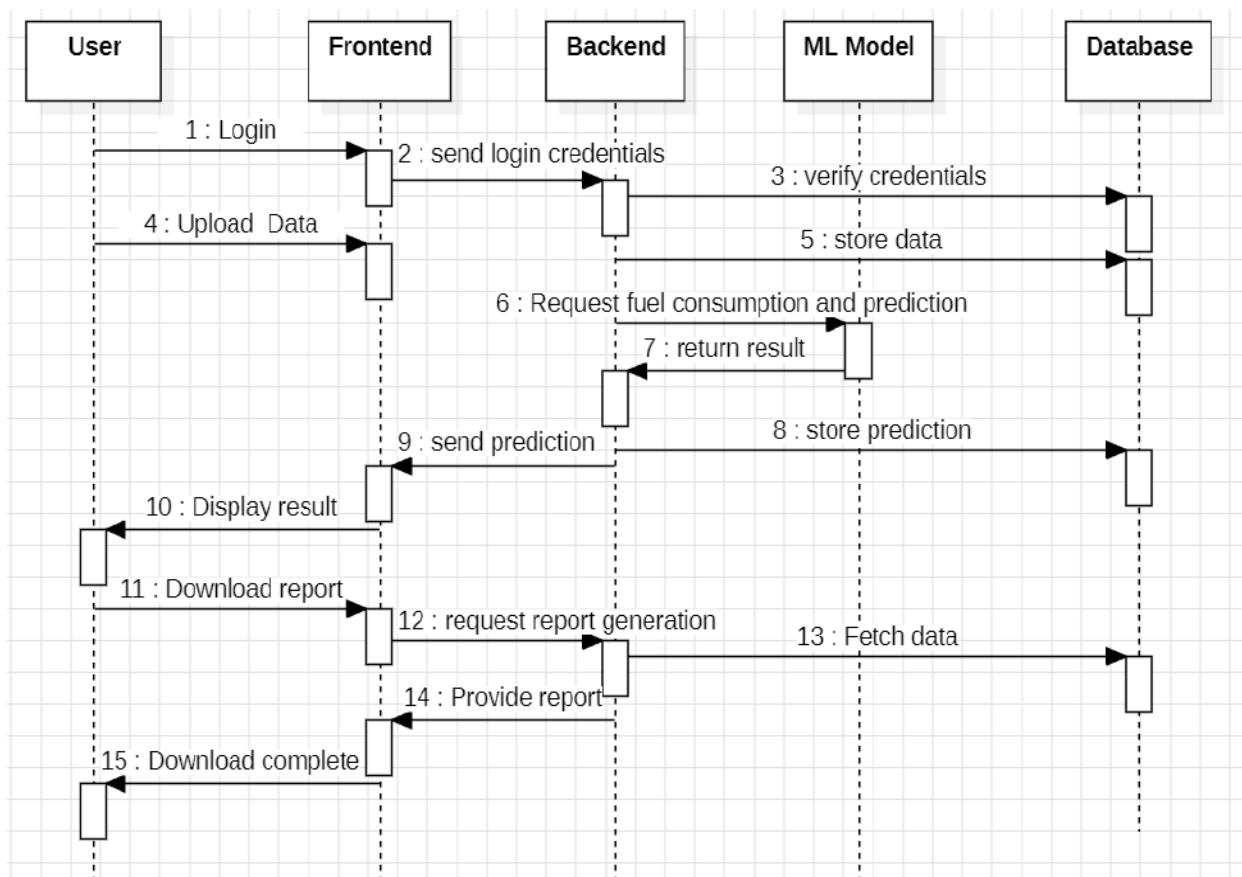


Fig 3.7 Sequence Diagram

3.3.5 Collaboration Diagram

A Collaboration Diagram (also called Communication Diagram) shows the structural organization of objects and their message passing relationships.

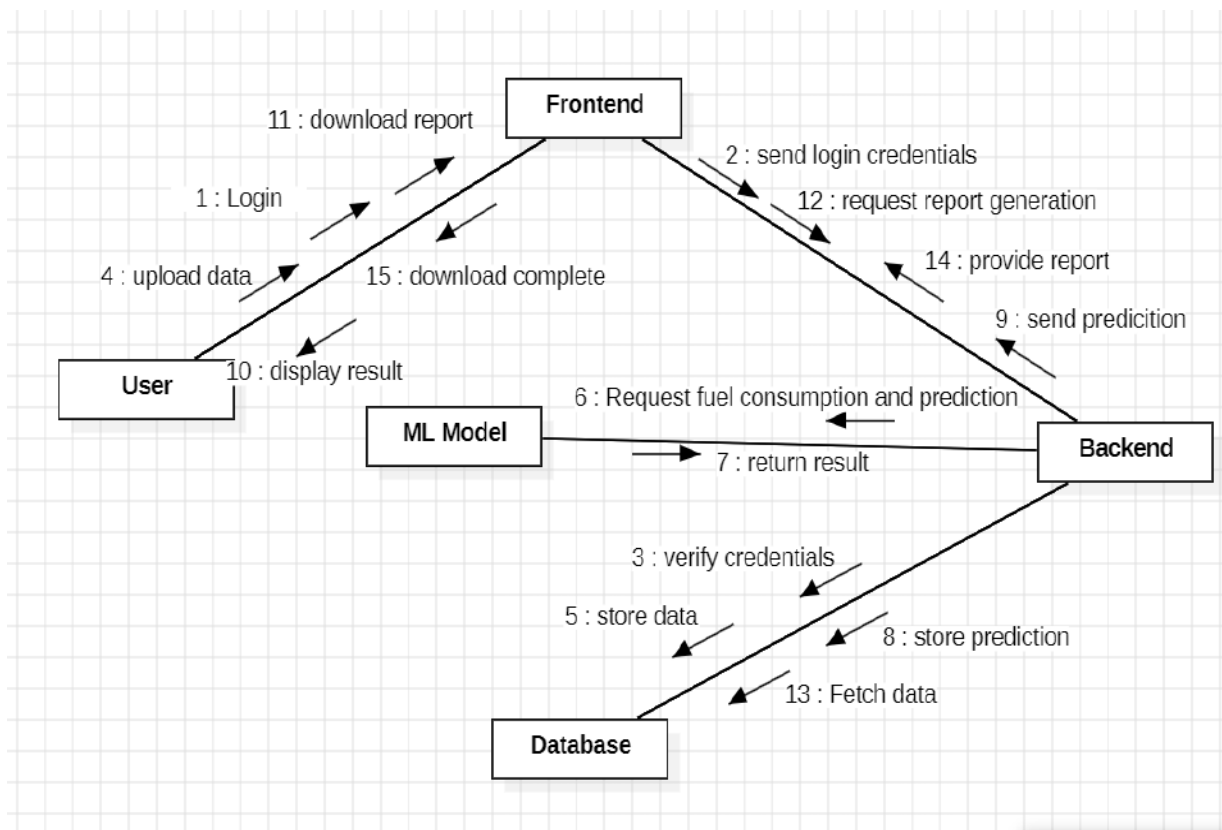


Fig 3.8 Collaboration Diagram

IMPLEMENTATION

4 IMPLEMENTATION

4.1 Technologies

This system is developed using PYTHON programming language.

4.1.1 Python

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. Created by Guido van Rossum and first released in 1991, Python is widely used for tasks ranging from web development and data analysis to artificial intelligence and automation.

4.1.2 Features of Python

Easy to Learn and Use: Python has a clean and straightforward syntax, making it accessible to beginners.

Versatile: Supports multiple programming paradigms like object-oriented, procedural, and functional programming.

Extensive Libraries: Python has a vast standard library and a rich ecosystem of third-party packages.

Cross-Platform: Runs on various operating systems, such as Windows, macOS, and Linux.

4.1.3 Applications of Python

Python's versatility makes it a go-to language in numerous domains:

Web Development: Frameworks like Django and Flask simplify building web applications, enabling developers to create scalable and secure websites.

Data Science and Analytics: Python's libraries like Pandas, NumPy, and Matplotlib make data visualization, and manipulation efficient.

Machine Learning and AI: Tools like TensorFlow, Keras, and Scikit-

learn empower developers to build predictive models and intelligent applications.

Automation and Scripting: Python excels at automating repetitive tasks, such as file organization, web scraping, and testing, saving time and effort.

Game Development: Libraries like Pygame allow developers to create engaging 2D games with ease.

Scientific Computing: Python supports scientific research and simulations through libraries like SciPy and SymPy.

Cyber Security: Python is widely used for building security tools, penetration testing, and analyzing vulnerabilities.

Embedded Systems and IoT: Frameworks like Micro-Python enable Python to run on micro-controllers, making it suitable for hardware projects.

4.1.4 Advantages of Python:

Ease of Learning: Python is often recommended as a first language due to its simplicity and beginner-friendly syntax.

Rapid Development: Python's high-level features and library support allow for quick prototyping and development.

Strong Community: Python's active community provides extensive resources, including tutorials, forums, and libraries.

Integration Capabilities: Python can easily integrate with other languages and technologies, such as C++, Java, and databases.

Scalability: Python is suitable for small scripts as well as large enterprise applications.

4.1.5 Disadvantages of Python

Performance: Python is slower than compiled languages like C++ or Java, making it less suitable for performance-critical applications.

Mobile Development: Python is not the preferred choice for building mobile applications, where languages like Swift or Kotlin dominate.

Runtime Errors: Python's dynamic typing can lead to runtime errors that might be missed during development.

Libraries used in python:

numpy - mainly useful for its N-dimensional array objects.

pandas - Python data analysis library, including structures such as data frames.

matplotlib - 2D plotting library producing publication quality figures.

4.2 Pseudo Code

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from catboost import CatBoostRegressor
import joblib
import os

# Load dataset
df = pd.read_csv('data/vehicles.csv', low_memory=False)

# Strip whitespaces from column names just in case df.columns = df.columns.str.strip()

# Select required columns
df = df[['displ', 'cylinders', 'make', 'fuelType', 'trany', 'city08']]

# Rename columns for better readability df.rename(columns={
'displ': 'Engine_Size', 'cylinders': 'Cylinders', 'make': 'Make', 'fuelType': 'Fuel_Type', 'trany':
'Transmission',
'city08': 'Fuel_Consumption'
}, inplace=True)

# Drop missing values df.dropna(inplace=True)

# Convert categorical features to category dtype categorical_cols = ['Make', 'Fuel_Type',
'Transmission']
for col in categorical_cols:
    df[col] = df[col].astype('category')
```

```
# Features and target
X = df[['Engine_Size', 'Cylinders', 'Make', 'Fuel_Type', 'Transmission']] y =
df['Fuel_Consumption']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train CatBoost model model = CatBoostRegressor(
iterations=1000, learning_rate=0.1, depth=6, verbose=100
)

model.fit( X_train, y_train,
cat_features=categorical_cols, eval_set=(X_test, y_test), use_best_model=True
)
# Save the model os.makedirs('models', exist_ok=True)
joblib.dump(model, 'models/catboost_model.pkl')

# Evaluate the model
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred)) r2 = r2_score(y_test, y_pred)

print("\n📊 Model Evaluation:") print(f'MAE: {mae:.2f}')
print(f'RMSE: {rmse:.2f}')
print(f'R²: {r2:.2f}')
```

TESTING

5 TESTING

5.1 Overview

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, subassemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

5.2 Dimensions of Testing

There are many different dimensions to consider:

1. Layers of the application (database, APIs, UI)
2. Scale of testing (unit, module, integration, scenario) Type of testing (functional, performance, security, etc.)
3. Methodology (exploratory, scripted manual, automated)

5.3 Stages of Testing

5.3.1 Unit Testing

During This first round of testing, the program is submitted to assessments that focus on specific units or components of the software to determine whether each one is fully functional. The main aim of this endeavour is to determine whether the application functions as designed.

In this phase, a unit can refer to a function, individual program or even a procedure, and White box testing method is usually used to get the job done. One of the biggest benefits of this testing phase is that it can be run every time a piece of code is changed, allowing issues to be resolved as quickly as possible. It quite common for software developers to perform unit tests before delivering software to testers for formal testing.

5.3.2 Integration Testing

Integration testing allows individuals the opportunity to combine all of the units within a program and test them as a group. This testing level is designed to find interface defects between the modules/functions. This is particularly beneficial because it determines how efficiently the units are running together. Keep in mind that no matter how efficiently each unit is running, if they properly integrated, it will affect the functionality of the software program. In order to run these types of tests, individuals can make use of various testing methods, but the specific method that will be used to get the job done will depend greatly on the way in which the units are defined.

5.3.3 System Testing

System testing is the first level in which the complete application is tested as a whole. The goal at this level is to evaluate whether the system has complied with all of the outlined requirements and to see that it meets Quality Standards. System testing is undertaken by independent testers who haven't played a role in developing the program. This testing is performed in an environment that closely mirrors production. System Testing is very important because it verifies that the application meets the technical, functional, and business requirements that were set by the customer.

5.3.4 Acceptance Testing

The final level, Acceptance testing (or User Acceptance Testing), is conducted to determine whether the system is ready for release. During the Software development life cycle, requirements changes can sometimes be misinterpreted in a fashion that does not meet the intended needs of the users. During this final phase, the user will test the system to find out whether the application meets their business needs. Once this process has been completed and the software has passed, the program will then be delivered to production. The extensiveness of these tests is just another reason why bringing software testers in early is important. When a program is more thoroughly tested, a greater number of bugs will be detected; this ultimately results in higher quality software.

5.4 Types of testing

5.4.1 Black box testing

It is also called as Behavioral/Specification-Based/Input-Output Testing. Black Box Testing is a software testing method in which testers evaluate the functionality of the software under test without looking at the internal code structure. This can be applied to every level of software testing such as Unit, Integration, System and Acceptance Testing.

5.4.2 White box testing

It is also called as Glass Box, Clear Box, Structural Testing. White Box Testing is based on applications internal code structure. In white-box testing an internal perspective of the system, as well as programming skills, are used to design test cases. This testing usually done at the unit level.

White Box Testing Techniques:

1. Statement Coverage
2. Branch Coverage
3. Path Coverage

5.5 Test Cases

Test Case ID	Input (Car Features)	Expected Output	Actual Output	Result
01	Cylinders: 6, Displacement: 200, Horsepower: 100, Weight: 2800, Model Year: 79	20-25 MPG	22 MPG	Pass
02	Cylinders: 4, Displacement: 107, Horsepower: 86, Weight: 2200, Model Year: 82	30-35 MPG	34 MPG	Pass
03	Cylinders: 8, Displacement: 350, Horsepower: 165, Weight: 4100, Model Year: 72	10-14 MPG	12 MPG	Pass

04	Cylinders: 4, Displacement: 98, Horsepower: 90, Weight: 2000, Model Year: 83	33–38 MPG	36 MPG	Pass
05	Cylinders: 5, Displacement: 150, Horsepower: 95, Weight: 2600, Model Year: 81	22–28 MPG	25 MPG	Pass
06	Cylinders: 3, Displacement: 70, Horsepower: 60, Weight: 1800, Model Year: 85	40–45 MPG	43 MPG	Pass

07	Cylinders: 6, Displacement: 232, Horsepower: 100, Weight: 3300, Model Year: 76	16–22 MPG	20 MPG	Pass
08	Cylinders: 4, Displacement: 112, Horsepower: 85, Weight: 2500, Model Year: 84	30–35 MPG	32 MPG	Pass
09	Cylinders: 8, Displacement: 360, Horsepower: 170, Weight: 4200, Model Year: 71	8–12 MPG	11 MPG	Pass
10	Cylinders: 4, Displacement: 121, Horsepower: 78, Weight: 2400, Model Year: 80	27–32 MPG	30 MPG	Pass

SCREENSHOTS

6 SCREENSHOTS

```
PS C:\Users\bende\Downloads\MPML 19\MINI_PROJECT> cd ".\ML19\CODE"
>> python app.py
>>
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
```

Fig 6.1 Activate Project

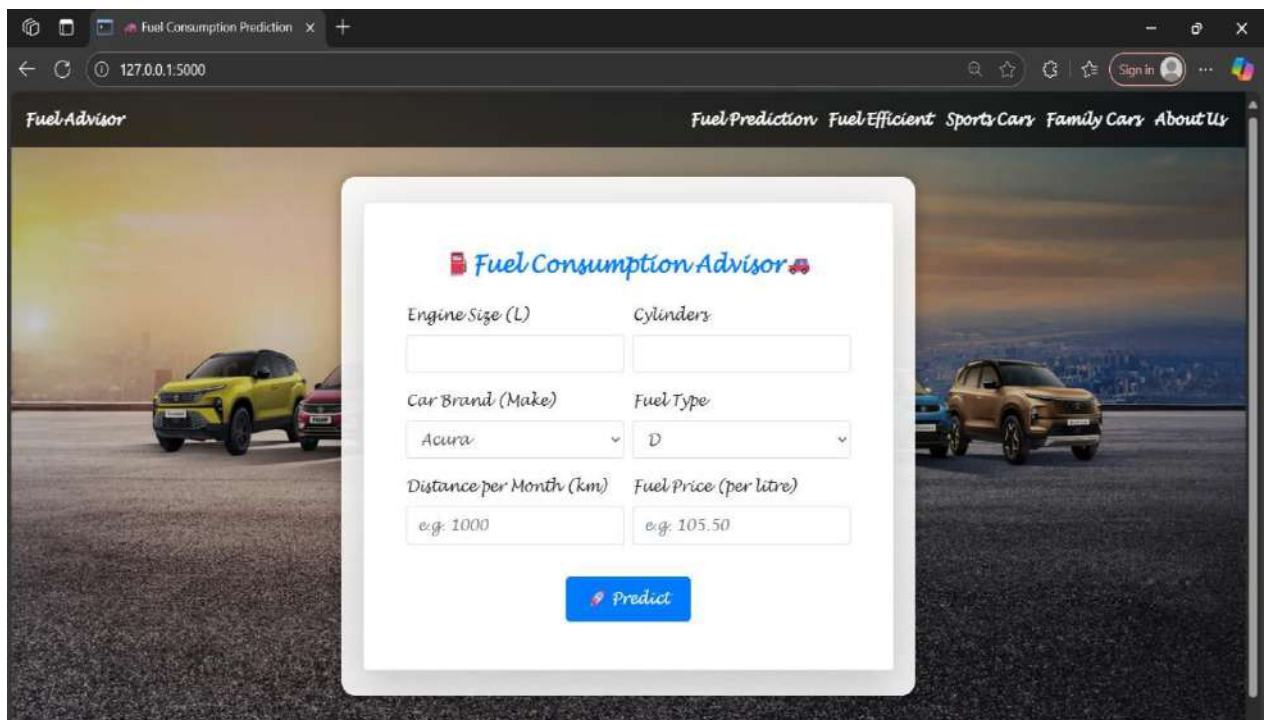


Fig 6.2 Main Page

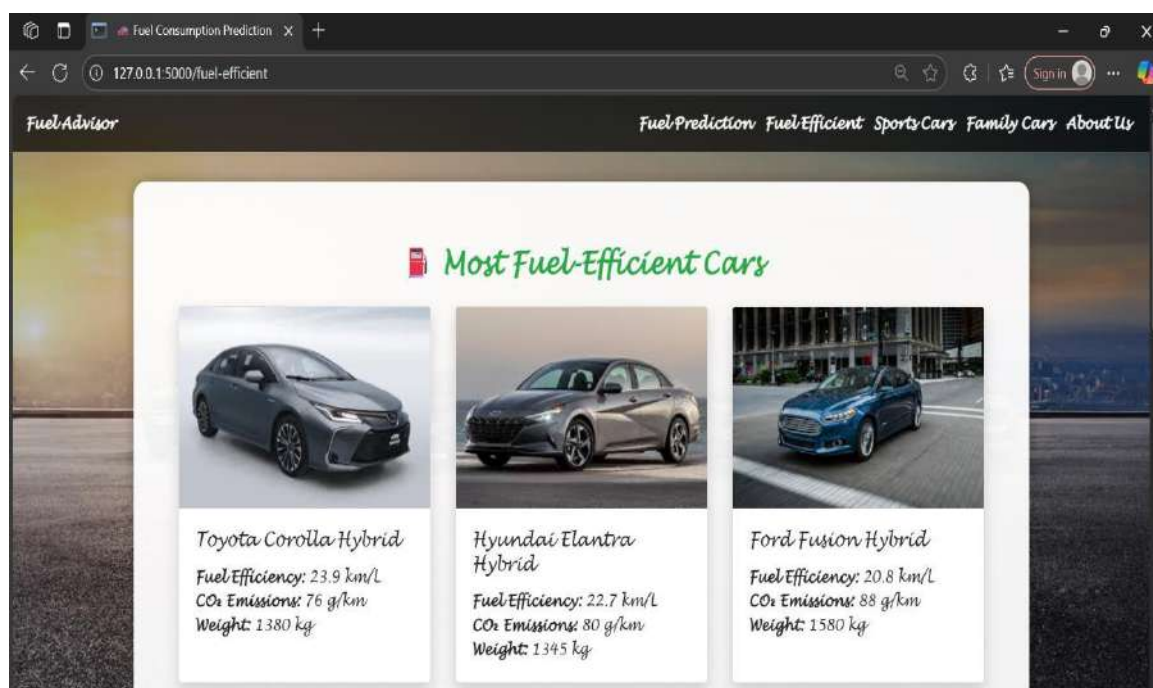


Fig 6.3 Fuel Efficient Cars

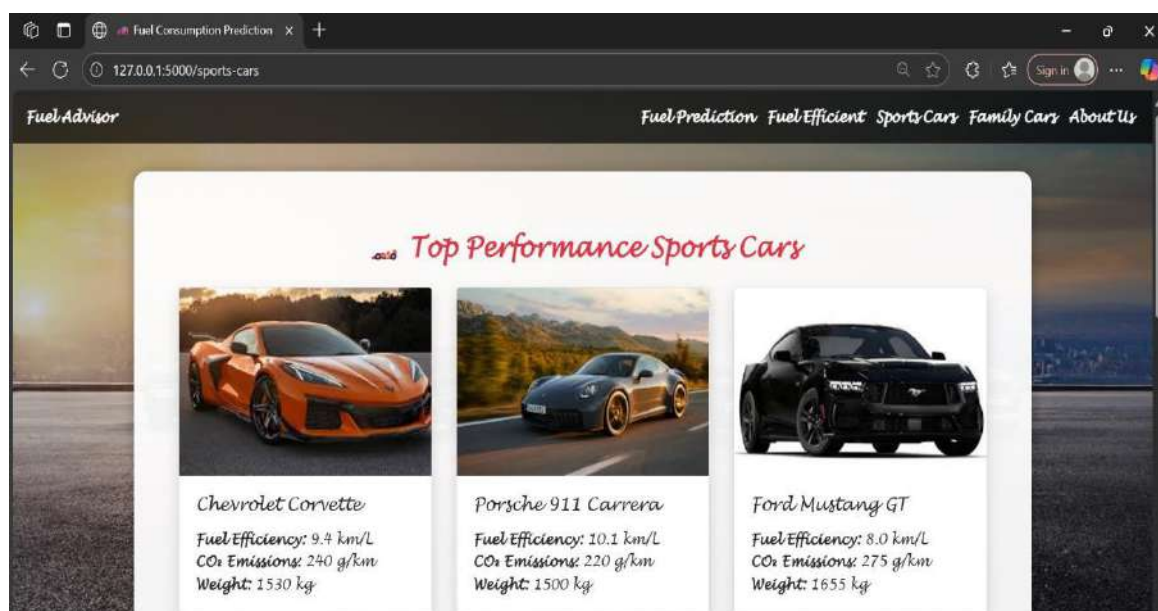


Fig 6.4 Sports Cars

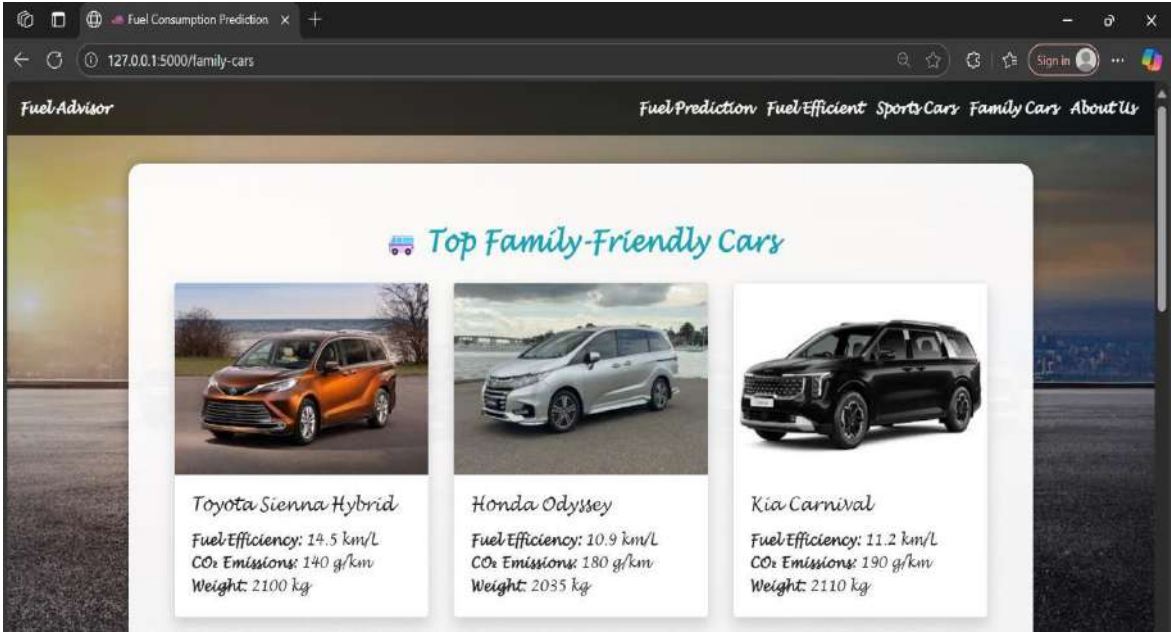


Fig 6.5 Family Cars

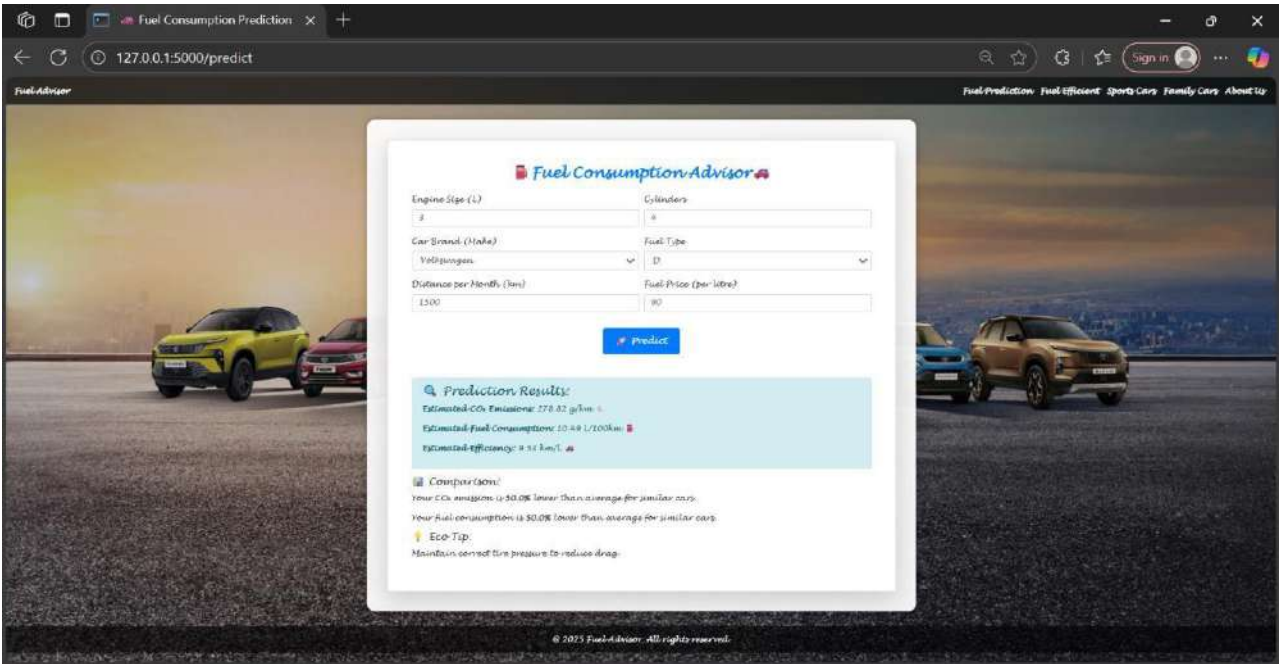


Fig 6.6 Output-1

The screenshot shows a web browser window with the URL `127.0.0.1:5000/predict`. The page features a dark background with a car image and a central white form titled "Fuel Consumption Advisor". The form contains the following fields and values:

Engine Size (L)	Cylinders
1.8	4

Car Brand (Make)	Fuel Type
Ford	E

Distance per Month (km)	Fuel Price (per litre)
1000	90

A blue "Predict" button is located below the input fields.

Prediction Results:

- Estimated CO₂ Emissions: 166.11 g/km
- Estimated Fuel Consumption: 7.43 L/100km
- Estimated Efficiency: 13.42 km/L

Comparison:

- Your CO₂ emissions is 87.2% lower than average for similar cars.
- Your fuel consumption is 87.7% lower than average for similar cars.

Eco Tip:

Reduce excess weight from your car.

© 2025 FuelAdvisor. All rights reserved.

Fig 6.7 Output-2

CONCLUSION AND FUTURE SCOPE

7. CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project, introduces a machine learning model designed to predict fuel consumption for heavy vehicles using seven key predictors derived from vehicle speed and road grade data. Among these predictors, the change in kinetic and potential energy are newly introduced to better capture the vehicle's dynamic behavior. All inputs can be obtained through telematics devices commonly found in connected vehicles, and computations can be performed onboard. The model aggregates predictor data over fixed distance windows instead of time intervals, which aligns with the target fuel consumption metric and improves prediction accuracy.

The model achieved high accuracy, with an RMSE of less than 0.015 l/100km and a coefficient of determination (CD) of 0.91 for 1 km window sizes. Comparisons across 1, 2, and 5 km windows showed the 1 km window performed best, particularly in mixed urban and highway driving conditions. Shorter windows are ideal for applications like construction fleets or city traffic, while longer windows may suffice for long-haul vehicles. The model outperforms previous ML approaches, which required entire trip data for similar results. Future work will focus on adapting the model to account for vehicle characteristics such as varying mass and wear over time.

7.2 Future Scope

Advanced Modeling Techniques

Explore more robust ML models like Gradient Boosted Trees, LSTMs, or Transformer-based models for better temporal dependency modeling.

Incorporate Additional Vehicle Parameters

Include predictors like vehicle mass, engine age, tire condition, and load weight. This can improve prediction accuracy across different vehicle types and operational conditions.

REFERENCES

REFERENCES

1. **Pereira, C. A., & Farias, T. L. (2020).**
Fuel consumption prediction for heavy-duty vehicles using machine learning: A case study.
Energy, 202, 117702.
2. **Zhao, Y., Burke, M., & Yildiz, Y. (2019).**
Physics-informed machine learning approach for fuel consumption prediction. *IEEE Transactions on Intelligent Transportation Systems*, 21(12), 4975–4984.
3. **Kamrani, M., Mahmassani, H. S., & Saberi, M. (2015).**
Modelling fuel consumption and emissions for urban transportation networks: A hybrid approach.
Transportation Research Part D: Transport and Environment, 37, 123–136.
4. **Yuksel, B., & Can, A. (2019).**
A novel model for estimating fuel consumption and emissions considering road surface conditions.
Sustainable Cities and Society, 51, 101764.